

OpenStack Advanced

Part 1 of 2

*Infrastructure as Code:
Heat & Terraform*

Cloud Computing

ADSAI – University of Trieste

Academic Year 2025/2026

Topics covered: IaC imperative • Heat architecture + HOT templates • ResourceGroup
• Heat CLI • Terraform OpenStack provider • Complete cluster HCL

Prerequisites: Lecture OpenStack Fundamentals openrc.sh configured for CINECA ADA

The Problem: Why Manual Provisioning Fails

You can type 25 CLI commands — but can you prove you typed them right six months later?

The Manual Provisioning Scenario

- You deploy a 4-node BLAST cluster: network, security groups, VMs, volumes, floating IPs.
- The experiment produces results. Paper accepted. 🎉
- Six months later: reviewer requests independent replication.
- Question: can you recreate the EXACT environment?
- Answer: almost certainly not — without IaC.

Reproducibility

Shell history ≠ documentation. Version mismatches accumulate silently.

Version Control

You cannot git diff a manually configured environment. Drift is invisible.

Automation

CI/CD pipelines need programmatic provision/teardown, not human keystrokes.

Auditability

Grant agencies increasingly require infrastructure config alongside code + data.

Solution: Infrastructure as Code (IaC) — declarative text files that are version-controlled, reviewed, tested, and applied idempotently.

Declarative vs. Imperative IaC

Same concept as Kubernetes control loop — describe desired state, let the tool figure out how

Imperative

"Do this, then that"

Example: Shell scripts
Ansible tasks
openstack server create (one by one)

- Order-dependent — partial failure leaves undefined state
- Requires explicit error handling at each step
- No idempotency — running twice creates duplicates
- Hard to review: procedure, not intent

Declarative

"This is the desired state"

Example: Heat HOT templates
Terraform HCL
Kubernetes YAML (from L05)

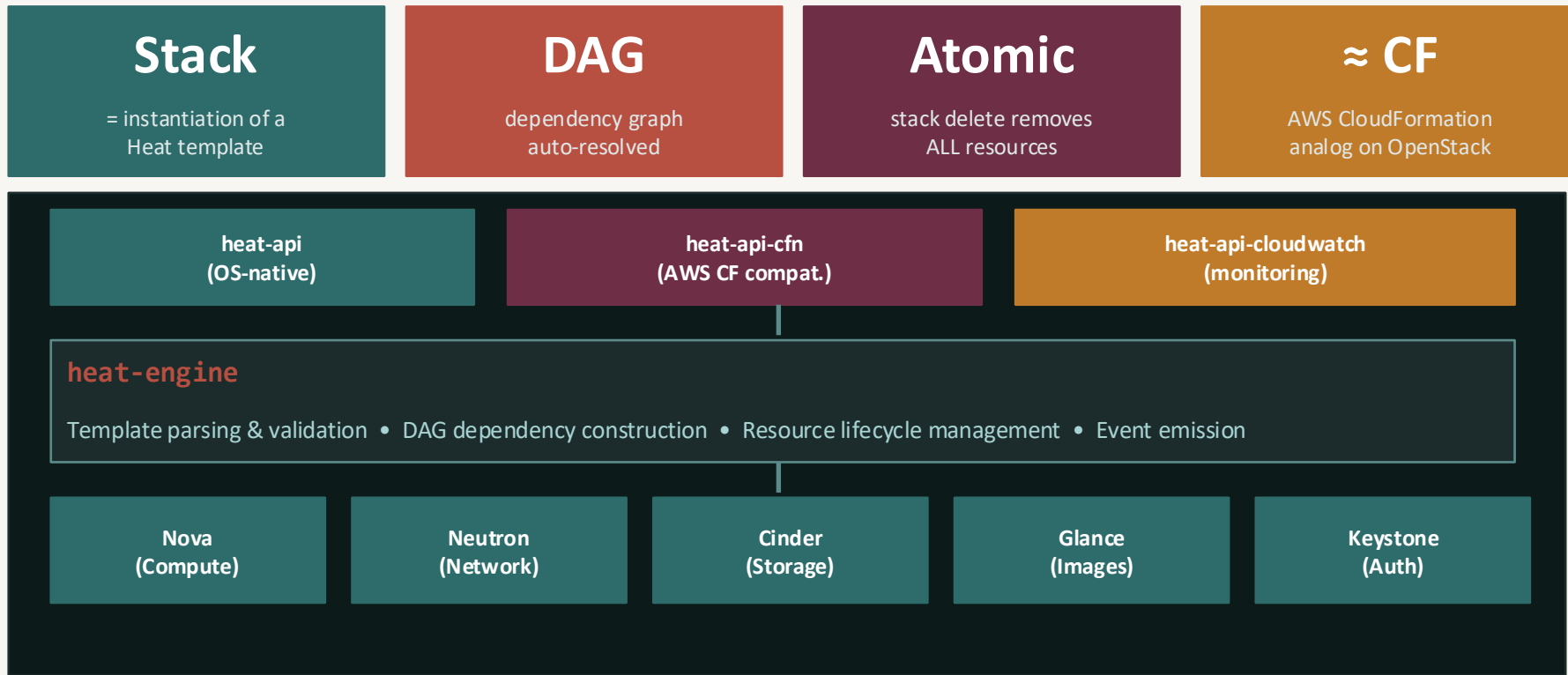
- Idempotent — running twice is safe; no duplicates
- Tool resolves dependency order automatically (DAG)
- State tracking — tool knows what already exists
- Reviewable: describes intent, not procedure

```
#!/bin/bash
openstack network create ...
openstack subnet create ...
openstack router create ...
openstack server create ...
# 21 more commands ...
```

```
# heat_template.yaml
resources:
  network:
    type: OS::Neutron::Net
  server:
    type: OS::Nova::Server
```

Heat: OpenStack-Native Orchestration

Modeled on AWS CloudFormation — DAG-based resource dependency resolution



DAG ordering: networks → subnets → routers → instances. Heat provisions in topological order — unlike sequential shell scripts.

HOT Template Anatomy: Five Top-Level Sections

Heat Orchestration Template — YAML with well-defined structure

Template Skeleton

```
heat_template_version: 2021-04-16

description: |
  What this stack does

parameters:          # ① user-supplied
  key_name:
    type: string
    default: my-key

conditions:          # ② boolean guards
  is_prod: {equals: [env, prod]}

resources:            # ③ CORE section
  my_net:
    type: OS::Neutron::Net
  my_server:
    type: OS::Nova::Server
    depends_on: [my_net]

outputs:              # ④ exposed values
  server_ip:
    value: {get_attr: [my_server,
                      first_address]}
```

① parameters

User-supplied variables at deploy time. Types: string, number, json, comma_delimited_list, boolean. Constraints: allowed_values, range, length.

② conditions

Boolean expressions evaluated at runtime. Used for conditional resource creation. Example: create extra storage only in production.

③ resources

CORE section — the actual OpenStack resources to create. Each has type (OS::Service::Resource), properties, and optional depends_on.

④ outputs

Values exposed after stack creation: IPs, SSH commands, URLs. Readable with openstack stack output show.

HOT Intrinsic Functions: Dynamic Value Resolution

Functions evaluated at runtime by heat-engine — glue between template sections

Function	Syntax	Purpose	Example
get_param	{ get_param: key_name }	Read a template parameter	Inject keypair name into Nova server
get_resource	{ get_resource: my_net }	Get the UUID of another resource	Attach server to the network created in same stack
get_attr	{ get_attr: [server, first_address] }	Read an attribute from a resource	Retrieve assigned private IP for NFS mount
str_replace	{ str_replace: {template:"...",params:{}} }	String interpolation	Build SSH command with actual floating IP
list_join	{ list_join: [",", [a,b,c]] }	Join list → string	Pass comma-separated host list to user_data
if	{ if: [cond_name, val_T, val_F] }	Conditional value selection	Use prod flavor in prod, dev flavor otherwise

get_resource — link resources in the same stack

```
private_subnet:
  type: OS::Neutron::Subnet
  properties:
    network: {get_resource: private_net}
    #           ↑ resolves to UUID at deploy time
```

str_replace — build dynamic user_data scripts

```
user_data:
  str_replace:
    template: |
      mount $HEAD_IP:/shared /shared
    params:
      $HEAD_IP: {get_attr:[head,first_address]}
```

HOT Example (1/2): Network + Security Group Resources

cluster.yaml — research cluster on CINECA ADA Cloud

Parameters + Network

```
heat_template_version: 2021-04-16

parameters:
  key_name:
    type: string
  worker_count:
    type: number
    default: 4
    constraints:
      - range: {min:1, max:32}
  external_network:
    type: string
    default: external-network

resources:
  private_net:
    type: OS::Neutron::Net
  private_subnet:
    type: OS::Neutron::Subnet
    properties:
      network: {get_resource: private_net}
      cidr: 10.10.0.0/24
  router:
    type: OS::Neutron::Router
    properties:
      external_gateway_info:
        network: {get_param: external_network}
```

Security Groups

```
head_sg:
  type: OS::Neutron::SecurityGroup
  properties:
    name: cluster-head-sg
    rules:
      - direction: ingress
        protocol: tcp
        port_range_min: 22
        port_range_max: 22
        remote_ip_prefix: 0.0.0.0/0

worker_sg:
  type: OS::Neutron::SecurityGroup
  properties:
    rules:
      - direction: ingress
        protocol: any
        remote_mode: remote_group_id
        remote_group:
          {get_resource: head_sg}
        # ← intra-cluster by SG ref
      - direction: ingress
        protocol: any
        remote_mode: remote_group_id
        # ← worker-to-worker (MPI)
```

HOT Example (2/2): Head Node, ResourceGroup, Outputs

cluster.yaml — continued. ResourceGroup = Heat's count pattern

Head Node + Floating IP

```
head_node:
  type: OS::Nova::Server
  properties:
    image: Ubuntu 22.04
    flavor: {get_param: head_flavor}
    key_name: {get_param: key_name}
    security_groups:
      - {get_resource: head_sg}
    user_data_format: RAW
    user_data: |
      #!/bin/bash
      apt-get install -y nfs-kernel-server
head_fip:
  type: OS::Neutron::FloatingIP
head_fip_assoc:
  type: OS::Neutron::FloatingIPAssociation
```

ResourceGroup + Outputs

```
worker_nodes:
  type: OS::Heat::ResourceGroup
  properties:
    count: {get_param: worker_count}
    resource_def:
      type: OS::Nova::Server
      properties:
        name:
          str_replace:
            template: worker-%index%
        user_data:
          str_replace:
            template: mount $HEAD:/shared
          params:
            $HEAD:{get_attr:[head,first_address]}
```

Key pattern: OS::Heat::ResourceGroup

Creates N identical resources. Uses %index% for unique naming. Equivalent to Terraform count meta-argument (L08). Heat resolves dependencies of each member independently — workers wait for head_node before mounting NFS.

Heat CLI: Stack Lifecycle Management

openstack stack ... — validate, create, monitor, update, delete

0. Validate

1. Create

2. Monitor

3. Update

4. Delete

0. Validate template without creating any resource

```
openstack stack create --dry-run -t cluster.yaml test-validate
```

1. Create stack – pass parameters on the command line

```
openstack stack create -t cluster.yaml \  
  --parameter key_name=my-keypair \  
  --parameter worker_count=4 \  
  --parameter image_name='Ubuntu 22.04' \  
  research-cluster
```

2. Monitor – watch resources and events

```
openstack stack show research-cluster  
openstack stack resource list research-cluster  
openstack stack event list research-cluster
```

3. Update – scale from 4 to 8 workers (rolling update)

```
openstack stack update -t cluster.yaml --parameter worker_count=8 research-cluster
```

4. Delete – atomically removes ALL resources in the stack

```
openstack stack delete research-cluster
```

⚠ This removes networks, floating IPs, and volumes too!

Stack states: *CREATE_IN_PROGRESS* → *CREATE_COMPLETE* | *UPDATE_IN_PROGRESS* → *UPDATE_COMPLETE* | *DELETE_IN_PROGRESS* → (gone). Inspect failures with *openstack stack failures list <name>*.

Heat Stack States and Debug Commands

State machine + diagnostic CLI — essential for production deployments

Stack State Machine			Diagnostic CLI Reference	
State	Meaning	Action	<pre># Show stack status + outputs openstack stack show research-cluster # List all resources with their states openstack stack resource list \ research-cluster # Watch events in real time (--follow) openstack stack event list \ research-cluster --nested-depth 2 # Inspect failures (most useful!) openstack stack failures list \ research-cluster # Read stack output after CREATE_COMPLETE openstack stack output show \ research-cluster head_node_ip # Abandon stuck stack (last resort) openstack stack abandon research-cluster</pre>	
CREATE_IN_PROGRESS	Heat is provisioning resources	Wait — check event list		
CREATE_COMPLETE	All resources created OK	SSH in, run workload		
CREATE_FAILED	At least one resource failed	stack failures list → fix → retry		
UPDATE_IN_PROGRESS	Rolling update in progress	Wait — avoid concurrent changes		
UPDATE_COMPLETE	Update applied successfully	Verify new resources		
UPDATE_FAILED	Update failed, may be partial	stack failures list → rollback		
DELETE_IN_PROGRESS	Tearing down all resources	Wait — do not interrupt		
DELETE_COMPLETE	Atomic deletion complete	No orphaned resources	If stuck in *_IN_PROGRESS for >10 min: check heat-engine logs (admin) or openstack stack failures list. Never ctrl+C a running apply — it does NOT cancel the operation.	

Terraform on OpenStack: The Multi-Cloud Advantage

Same HCL, same workflow, same state model — applied to private cloud

You already know Terraform (L08 — AWS). The OpenStack provider gives you the same declarative workflow on CINECA ADA. Skills earned here transfer directly to AWS, Azure, GCP. One tool, every cloud.

Why Terraform on OpenStack?

- HCL syntax is identical to L08 AWS examples
- count / for_each for multi-node clusters
- terraform plan shows drift before apply
- terraform destroy removes everything cleanly
- State file enables incremental updates
- Modules reusable across OpenStack + AWS

Provider Config

- Source: terraform-provider-openstack/openstack
- Version: ~> 1.54 (stable for Wallaby)
- Auth: reads OS_* env vars from openrc.sh
- No secrets in HCL — all from environment
- cacert_file for CINECA TLS chain
- terraform init downloads provider automatically

Workflow Recap (L08)

- terraform init — download providers
- terraform plan — preview changes
- terraform apply — create/update
- terraform destroy — tear down all
- terraform.tfstate — tracks deployed resources
- Remote state: S3 / OpenStack Swift backend

```
provider "openstack" { cacert_file = "/path/to/cineca-chain.pem" } # versions.tf: source = "terraform-provider-openstack/openstack", version = "~> 1.54"
```

Terraform Provider Config and Data Sources

Data sources query existing infrastructure — resources create new ones

versions.tf + provider.tf

```
# versions.tf
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    openstack = {
      source = "terraform-provider-openstack/openstack"
      version = "~> 1.54"
    }
  }
}

# provider.tf — reads OS_* env vars from openrc.sh
provider "openstack" {}
```

Data Sources — query existing resources

```
# Read existing image from Glance
data "openstack_images_image_v2" "ubuntu" {
  name      = "Ubuntu 22.04"
  most_recent = true
}

# Read provider (external) network
data "openstack_networking_network_v2" "ext" {
  name = "external-network"
}

# Read a flavor by name
data "openstack_compute_flavor_v2" "medium" {
  name = "m1.medium"
}
```

data source vs resource — the key distinction

	data source	resource
Declaration	<code>data "type" "name" { ... }</code>	<code>resource "type" "name" { ... }</code>
Effect	Read-only — queries existing infra	Creates / updates / destroys infra
Lifecycle	No state owned by Terraform	Tracked in terraform.tfstate
Use case	Image ID, external network, flavor	Network, VM, floating IP, volume
AWS analog	<code>aws_ami</code> (data)	<code>aws_instance</code> (resource)

Terraform Example (1/3): Network and Security Groups

main.tf — same cluster as the HOT example, now in HCL

Network Infrastructure

```
resource "openstack_networking_network_v2" "cluster" {
  name          = "cluster-network"
  admin_state_up = true
}

resource "openstack_networking_subnet_v2" "cluster" {
  network_id =
  openstack_networking_network_v2.cluster.id
  cidr       = "10.10.0.0/24"

  dns_nameservers = ["8.8.8.8", "8.8.4.4"]
}

resource "openstack_networking_router_v2" "router" {
  external_network_id =
  data.openstack_networking_network_v2.ext.id
}

resource "openstack_networking_router_interface_v2" "iface" {
  router_id = openstack_networking_router_v2.router.id
  subnet_id = openstack_networking_subnet_v2.cluster.id
}
```

Security Groups

```
resource "openstack_networking_secgroup_v2" "head" {
  name = "cluster-head-sg"
}

resource "openstack_networking_secgroup_rule_v2" "ssh" {
  direction      = "ingress"
  protocol       = "tcp"
  port_range_min = 22
  port_range_max = 22
  remote_ip_prefix = "0.0.0.0/0"
  security_group_id = openstack_networking_secgroup_v2.head.id
}

resource "openstack_networking_secgroup_v2" "workers" {
  name = "cluster-worker-sg"
}

resource "openstack_networking_secgroup_rule_v2" "intra" {
  direction      = "ingress"
  remote_group_id =
  openstack_networking_secgroup_v2.workers.id
  security_group_id =
  openstack_networking_secgroup_v2.workers.id
}
```

Terraform Example (2/3): Head Node, Workers with count, Volume

count meta-argument — Heat's OS::Heat::ResourceGroup analog

Head Node + Floating IP

```
resource "openstack_compute_instance_v2" "head" {
  name      = "cluster-head"
  image_id  = data.openstack_images_image_v2.ubuntu.id
  flavor_id = data.openstack_compute_flavor_v2.medium.id
  key_pair  = openstack_compute_keypair_v2.kp.name
  security_groups =
[openstack_networking_secgroup_v2.head.name,
openstack_networking_secgroup_v2.workers.name]
  network { uuid = openstack_networking_network_v2.cluster.id
}
  user_data = <<-EOF
    #!/bin/bash
    apt-get install -y nfs-kernel-server
    echo "/shared *(rw,sync)" >> /etc/exports
    EOF
}
```

Worker Nodes (count) + Shared Volume

```
# count creates N identical resources
resource "openstack_compute_instance_v2" "workers" {
  count      = var.worker_count
  name      = "cluster-worker-${count.index + 1}"
  image_id  = data.openstack_images_image_v2.ubuntu.id
  flavor_name = var.worker_flavor
  depends_on = [openstack_compute_instance_v2.head]
  network { uuid = openstack_networking_network_v2.cluster.id
}
}
resource "openstack_blockstorage_volume_v3" "data" {
  name = "cluster-shared-data"
  size = 500
}
resource "openstack_compute_volume_attach_v2" "attach" {
  instance_id = openstack_compute_instance_v2.head.id
  volume_id   = openstack_blockstorage_volume_v3.data.id
}
```

count vs. for_each: count = var.worker_count creates resources indexed 0..N-1, named via \${count.index}. Use for_each when nodes have different configs. depends_on ensures workers start only after head node is ACTIVE — NFS must be ready.

Terraform Example (3/3): Outputs and MPI Hostfile Generation

outputs.tf + local_file resource — infrastructure outputs drive workflow

outputs.tf

```
output "head_public_ip" {
  value = openstack_networking_floatingip_v2.head_fip.address
}

output "worker_private_ips" {
  value =
  openstack_compute_instance_v2.workers[*].access_ip_v4
}

output "ssh_command" {
  value = "ssh
ubuntu@${openstack_networking_floatingip_v2.head_fip.address}
"
```

MPI Hostfile Generation

```
# Generate hostfile from worker IPs
resource "local_file" "mpi_hostfile" {
  content = join("\n", [
    for ip in
    openstack_compute_instance_v2.workers[*].access_ip_v4 :
    "${ip} slots=4"
  ])
  filename = "hostfile"
}

output "mpi_run_command" {
  value = "mpirun --hostfile hostfile -np ${var.worker_count *
4} ./sim"
}
```

Applying the configuration

```
source openrc.sh # Load credentials into OS_* env vars
terraform init # Download openstack provider plugin
terraform plan -var='key_name=my-key' # Preview: 14 resources to add
terraform apply -var='key_name=my-key' # Create cluster (confirm with 'yes')
ssh ubuntu@$(terraform output -raw head_public_ip) # Connect!
terraform destroy -var='key_name=my-key' # ⚠️ Clean up when done
```

Heat vs. Terraform: When to Choose Which

Not personal preference — architectural implications for your research workflow

Dimension	Heat	Terraform
Native OpenStack integration	Yes — first-class citizen	Via provider plugin
Template language	YAML (HOT)	HCL
State management	OpenStack DB (heat-engine)	Local .tfstate or remote backend
Drift detection	Stack events, resource list	terraform plan shows exact drift
Multi-cloud support	OpenStack only	AWS, Azure, GCP, OpenStack, K8s, ...
Skill transferability	Limited to OpenStack	High — same tool across all clouds
CloudFormation compat.	Yes (heat-api-cfn endpoint)	N/A — different paradigm
Composability	Nested stacks, ResourceGroup	Modules, count, for_each
Rollback on failure	Automatic (configurable)	Manual (state manipulation needed)
Dry-run capability	--dry-run flag	terraform plan
Community / ecosystem	OpenStack community	Very large, multi-cloud

Recommendation for MHPC: use Terraform when workflows span multiple clouds (ADA + AWS). Use Heat when deep OpenStack lifecycle integration or CFn compatibility is required. Many teams use both.

State Management: The Critical Difference

Heat keeps state in OpenStack DB — Terraform in a file you must protect

Heat State Model

- State stored in OpenStack Heat database — not on your laptop
- Any team member can run `openstack stack show <name>`
- Concurrent operations protected by database transactions
- State persists across team members and CI/CD systems
- Deleting a stack = deleting the state; no orphan risk
- Downside: tied to this OpenStack installation

Terraform State Model

- `terraform.tfstate` is a JSON file — must be version-controlled
- Lost state = Terraform cannot manage existing resources
- Remote backends (S3, Swift) solve team collaboration
- State locking prevents concurrent apply conflicts
- `terraform.tfstate` contains secrets — must be encrypted at rest
- Advantage: works anywhere, not tied to OpenStack

```
# Anyone with access can query state:
openstack stack resource list cluster
openstack stack output show cluster head_ip
openstack stack event list cluster --nested-depth 2
```

```
# Remote state on Swift (object storage)
terraform {
  backend "swift" {
    container = "tf-state-research-cluster"
  }
}
```

⚠ Never commit `terraform.tfstate` to a public repository — it contains passwords, private IPs, and resource IDs. Use `.gitignore` or a remote backend.

Debugging IaC Deployments: Heat and Terraform

Systematic approach — both tools emit events/logs; learn to read them

Failure symptom	Heat diagnosis	Terraform diagnosis
Stack stuck / timeout	openstack stack event list cluster	TF_LOG=DEBUG terraform apply
Resource in ERROR state	openstack stack failures list cluster	terraform show (inspect state)
VM not reachable after create	openstack console log show <vm>	Same — check console_log output attr
Quota exceeded	openstack stack show → OVER_LIMIT	Error: 403 Over Limit — check quota show
Network not routable	openstack stack resource list → check router	terraform state show openstack_networking_router_v2.*
Floating IP not assigned	openstack floating ip list → fixed_ip_address NULL	check resource openstack_compute_floatingip_associate_v2
user_data script failed	openstack console log show <vm> → cloud-init output	Same — user_data in Terraform is identical

Universal debug order (both tools):

① Check stack/plan output for the first error → ② Identify which resource failed → ③ console log show for VM issues → ④ openstack quota show for 403s → ⑤ security group rules for connectivity

Part 1 — Key Takeaways

IaC Imperative

Shell history ≠ documentation. Reproducibility, version control, and auditability require declarative text files that can be git-committed and peer-reviewed.

Declarative Model

Both Heat and Terraform describe desired state. The tool builds a DAG, resolves dependencies, and applies changes idempotently. Running twice is safe.

Heat Architecture

heat-engine parses HOT → builds DAG → provisions resources in topological order. Stacks are first-class objects; atomic deletion prevents orphaned resources.

HOT Template

Five sections: parameters → conditions → resources → outputs. Intrinsic functions (get_resource, get_attr, str_replace) wire resources together at deploy time.

Terraform on OS

Same HCL, same workflow as AWS (L08). Data sources read existing infra; resources create new. count = N workers. State file must be protected.

Heat vs. Terraform

Heat: deeper OpenStack integration, automatic rollback. Terraform: multi-cloud, higher skill portability, richer ecosystem. Both are valid — context decides.